

An Explainable Rule-Based Approach to Automatic Rhythm Game Chart Generation

Abstract

Chart creation is a central content-creation task in rhythm-based music games (DJMAX, Project Sekai, etc.): the note sequences players hit are traditionally authored by hand by expert charters, making the process labor-intensive and heavily dependent on individual expertise. We present **Rhythm Chart Generator**, a system that automatically produces 4–8 key charts from arbitrary audio (a local file or a YouTube link). It (i) estimates BPM, beats, section structure, instrument focus, and seven musical feature vectors with a lightweight NumPy signal-processing pipeline; (ii) places notes using a feature-based pattern-scoring scheduler, contour-based motif memory, and a closed-loop density controller; and (iii) scores each result on six quality dimensions and selects the best via multi-seed search. Rather than a learned black box, the proposed system encodes musical principles as explicit rules: it is reproducible with a fixed random seed while allowing controlled probabilistic variation, and it records post-hoc, traceable rationale for every note. Without copying any charter’s work, it targets conformance to design principles common across commercial games. The system runs in pure Python/NumPy with no GPU or training data, processing a 3.5-minute song in about 1.4 seconds, and is released as an open-source desktop and web application.

Keywords: Rhythm game; Chart generation; Music information retrieval (MIR); Procedural generation; Explainable systems

1. Introduction

1.1 Background

In rhythm games the chart is the core piece of game content, and producing it automatically is a procedural content generation (PCG) problem specific to music games. Even for the same song, the player experience varies greatly with chart quality. A good chart has the following properties.

- **Rhythmic fit:** notes coincide with the song’s actual attack points (kick, snare, hi-hat, vocal onsets).
- **Natural hand movement:** the two hands alternate, without meaningless jumps or excessive same-lane repetition (jacks).
- **Structural awareness:** it follows a difficulty arc — verses light, choruses dense, bridges as rest sections.
- **Balance of diversity and repetition:** patterns that are neither fully random nor fully repetitive — "familiar yet fresh."

Reproducing these properties automatically requires more than simply placing a note at every onset. One must understand the song’s structure, follow the melody contour, model the physical motion of the

hands, and remember recurring phrases to vary them — porting the expert charter's thought process into an algorithm.

1.2 Problem Definition

The input is an audio signal $x(t)$ and target parameters (number of keys $K \in \{4, 5, 6, 7, 8\}$, difficulty $d \in \{\text{easy, normal, hard, expert, master, style } S\}$). The output is a note sequence

$$C = \{(t_i, \ell_i, k_i, \delta_i)\}_{i=1}^N,$$

where t_i is the note time, $\ell_i \in \{0, \dots, K - 1\}$ the lane, $k_i \in \{\text{tap, hold}\}$ the kind, and δ_i the hold length. The goal is to find C that maximizes the quality properties of §1.1.

1.3 Approach and Contributions

The system is **rule-based rather than learning-based**. This is because (1) public training data for rhythm-game charts is scarce and formats are fragmented across games; (2) learning from a specific charter's work raises copyright and copying concerns; and (3) a rule-based approach can explain the rationale of each decision and is therefore user-tunable.

A note on the scope of our claims. This paper does *not* claim to have "reproduced expert-charter quality as judged by human evaluation." A large-scale human player-preference study or a blind A/B comparison against human charts is beyond the scope of this work and is left as future work (§8.2). What this work demonstrates is strictly **conformance to predefined rules** (target density, hold ratio, hand movement, diversity/repetition, etc.). The observed commercial- game design principles are referenced only as the *design rationale* for the rules; whether those principles are perceptually reproduced for humans was not verified by qualitative evaluation.

This paper makes the following five contributions.

Contribution 1 — A lightweight MIR pipeline in pure NumPy, without GPU or deep learning. It extracts autocorrelation-based local BPM estimation and a tempo map, band separation, and seven chroma/spectral-feature-based musical feature vectors (vocal presence, drum fill, tension, etc.) at near-real-time speed (§4).

Contribution 2 — A feature-based pattern scoring scheduler. It applies a "generate–evaluate–select" strategy at the pattern level: generate many candidate patterns, score each against the live musical features, then select via softmax sampling (§5.2).

Contribution 3 — Contour-based motif memory and section replay. It recognizes phrases by pitch-interval contour rather than absolute pitch, so a melody that recurs in a different key still matches, and replays it with progressive transforms (identity → mirror → reverse) (§5.3).

Contribution 4 — A rhythm-fit-preserving closed-loop density controller. It feeds back the error between target density (NPS) and actual density to adjust the placement threshold, but multiple safeguards (onset floor, energy floor, upper-bound clamp) categorically prevent notes from being generated where there is no real attack. That is, it is designed so the density target cannot damage rhythmic fit (§5.4).

Contribution 5 — A six-dimensional quality evaluator with multi-seed auto-improvement. It scores the output quantitatively on rule-conformance criteria and selects the best of several

candidates, leaving a traceable report of the selection rationale. This evaluation is, as discussed in §8.2, explicitly a self-evaluation (§6).

2. Rhythm Game Chart Generation and Procedural Content Generation

Beat tracking and onset detection. BPM/beat estimation is a classic MIR problem; spectral-flux-based onset detection and autocorrelation/comb-filter tempo estimation are widely used. The system reproduces these classic techniques without a GPU, combining short-window local BPM estimation into a tempo map that captures tempo changes.

Source separation. Deep-learning stem separators such as Demucs and Spleeter are the standard but are heavy and require a GPU. The system uses a lightweight alternative that approximately classifies kick/snare/vocal/hi-hat using only low/mid/high frequency-band separation and spectral features (flatness, chroma, entropy).

Procedural / automatic chart generation. Existing automatic charting tools mostly either place a note at every onset or are deep-learning models specialized to a particular game format. The former ignores structure, hand movement, and diversity and thus feels "mechanical"; the latter has training-data and explainability issues. A representative learning-based prior work, **Dance Dance Convolution (DDC)** [1], learns the placement and selection of DDR steps with a CNN+LSTM, requiring a human-authored step-chart corpus and GPU training. The system differs in that it is rule-based yet integrates structural awareness, motif memory, and density control to **encode** the designer's principles **as rules**, and in that it runs on CPU alone without training, a GPU, or a corpus.

2.1 Compute and Speed Comparison

A key advantage of the rule-based approach is its **lightness and speed**. Table 1 compares the compute profile of the system with that of learning-based generators. For academic rigor, however, we **clearly distinguish measurement provenance**. The numbers for this work (★) were measured directly on a consumer CPU, whereas the deep-learning values are qualitative characteristics reported in the corresponding original papers/designs. We **did not conduct a controlled experiment directly comparing the different systems on the same hardware and the same songs** (due to the cost of reproducing the training and environment of the target systems); this is left as future work (§8.2). Table 1 should therefore be read not as a precise head-to-head benchmark but as a **paradigm-level comparison of resource requirements**.

Table 1. Compute profile comparison (★ = measured directly in this work, † = characteristics as reported in the original paper/design)

Item	This work (rule-based)	DDC [1] etc. (learning-based)
Paradigm	Rule-based (no training)	Supervised (CNN+LSTM) †
Training data	None ★	Human-authored chart corpus required †
GPU for training	None ★	Required (model training) †
Inference stage	CPU, NumPy only ★	Neural forward pass (GPU acceleration typical) †
Model weights	None (code + JSON patterns) ★	Trained parameters required †
Core dependencies	<code>numpy</code> (+ <code>yt-dlp</code> + <code>ffmpeg</code> for YouTube input) ★	Deep-learning framework (PyTorch/TF, etc.) †
Explainability	Post-hoc traceable (§8.1) ★	Black box †

Table 2. Measured performance of the system (★, single-threaded consumer CPU: AMD Ryzen family, Python 3.12 / NumPy 2.4, no GPU. Mean of 5 runs)

Song length	Analyze	Generate (hard)	Total	Real-time factor	Peak heap
24 s	71 ms	5.7 ms	76 ms	× 314	57 MB
210 s	963 ms	425 ms	1.39 s	× 151	~500 MB

The full pipeline (analyze + generate) processes a 3.5-minute song in about 1.4 seconds — roughly 150× faster than playback — and runs on a laptop-class CPU with no GPU or model weights. Memory scales linearly with song length (dominated by the spectrum arrays), about 0.5 GB for a 3.5-minute song, below the VRAM requirements of GPU models (on the order of several GB). Per-difficulty generation time scales with note density (easy 3 ms – master 19 ms, for 24 s). These figures can be re-measured with the reproduction procedure in §7.

Optimization note. The initial implementation kept the frame-time axis as a Python list and called `np.searchsorted` on every grid step, so generation scaled effectively quadratically for long songs (about 6.4 s for a 210 s song). Converting this axis to an ndarray once before the loop, making lookups $O(\log n)$, sped up generation by **about 15×** (6.4 s → 0.43 s), and the output is bit-for-bit identical (regression metrics unchanged).

3. System Overview

The overall pipeline consists of two stages.



The implementation is split into a backend (Python/NumPy, FastAPI) and a frontend (React/Vite/TypeScript); the desktop app starts a local server and opens the UI in an address-bar-less web-app window.

4. Audio Analysis Pipeline

`audio_pipeline.analyze()` returns an `AnalysisResult` containing: BPM, beat offset, beat grid, measures, onset strength, RMS, tempo map, melody contour, section structure, band energy, instrument focus segments, and seven feature vectors.

4.1 Preprocessing and Framing

Audio is merged to mono and normalized, then downsampled to 22,050 Hz and stride-framed with frame size 2048 and hop 512. A Hanning window is applied to each frame to compute RMS and the rFFT spectrum. Onset strength is defined as a weighted sum of spectral flux (the sum of positive spectral differences) and the RMS rise:

$$\text{onset}[n] = 0.75 \cdot \hat{\text{flux}}[n] + 0.25 \cdot (\Delta \hat{\text{RMS}})^+[n]$$

where $\hat{\cdot}$ denotes 5th–95th percentile normalization.

4.2 BPM, Beats, and Tempo Map

Local BPM is estimated from the highest autocorrelation lag of an onset segment, folded into the 70–210 BPM range to suppress octave errors. The beat length is obtained from the global BPM, and the beat phase (offset) is estimated as the median of the onset-peak times modulo the beat length. Re-estimating local BPM over 8-second windows with a 4-second hop yields a **tempo map** that emits a new `TempoPoint` only when the value shifts by more than 3 BPM from the previous segment. Fixed-tempo songs like EDM produce a single point; live/rubato material captures the drift.

4.3 Band Separation and Melody Contour

Frequencies are split into low (<250 Hz), mid (250–4000 Hz), and high (≥ 4000 Hz) bands, and per-frame band energy is computed. This is the basis for approximately classifying kick/snare/vocal/hi-hat without heavy source separation. The melody contour maps the spectral centroid in the 150–2500 Hz band onto a log-frequency axis and smooths it, expressed as a per-frame value from 0 (low) to 1 (high). This value is later used to tie lane placement to melody rise/fall.

4.4 Section Structure Detection

The energy of 4-second windows is normalized and first classified into intro/verse/chorus/bridge/outro, then adjacent identical sections are merged. Post-processing inserts a **pre-chorus** into the rising-energy region just before a chorus and re-labels high-energy, high-treble-dominant regions as a **solo**. This structure is used as context for the difficulty arc (§5.1) and pattern selection (§5.2).

4.5 Instrument Focus Detection

For each 3-second window, onset density, band ratios, sustain, and onset standard deviation are used as features to score drums/vocal/guitar/bass/keys, and the top-scoring instrument is assigned as the focus. Confidence is computed from the margin between the 1st and 2nd scores; if the margin is small the window is left as "mixed." Adjacent identical foci are merged. The focus instrument locally shifts the charting style (vocal regions $\uparrow$ holds and melody following, drum regions $\uparrow$ anchoring and trills), so the character of the chart changes naturally as the song progresses.

4.6 Extended Feature Vectors

Seven frame-aligned features are computed, all in $[0, 1]$: spectral flux, instrument-change rate (band-distribution change), harmonic/percussive ratio (based on spectral flatness), chord change (chroma novelty), vocal-presence probability, drum-fill probability, and tension (a weighted sum of rising energy + chroma entropy + density + flux). These vectors are fed directly to the §5.2 pattern scorer to judge whether the moment is a sustained vocal line, a drum fill, or a rising build-up, and thereby choose patterns matching the musical character.

5. Chart Generation Engine

5.1 Intensity Curve — the Difficulty Arc

An expert charter designs a "difficulty arc" across the whole song. `build_intensity_curve()` builds a continuous intensity curve by adding the following corrections to a per-section base intensity (intro 0.25 ... chorus 0.90 ... solo 1.00).

- **Section-recurrence bonus:** the 2nd chorus is stronger than the 1st (up to +0.15).
- **Global-progression bonus:** later sections are naturally stronger (up to +0.08).
- **Within-section ramp:** the pre-chorus rises smoothly into the chorus, the bridge rests early, the outro descends.

The intensity is linearly interpolated at any time and becomes the reference for the density target.

5.2 Feature-Based Pattern Scoring Scheduler

As one of the core contributions, instead of flat weighted-random selection, this uses a pipeline (`_candidate_search`) of **generate many candidates** → **feature scoring** → **keep top-K** → **softmax final pick**.

1. **Generate.** From the inline pattern library (29 types: stairs, trills, jacks, bursts, holds, spirals, swings, etc.) and JSON patterns, candidates are filtered by the current section, difficulty, intensity, and style conditions. Each candidate spawns 1–3 variations via spatial transforms (identity/mirror/reverse), expanding the pool.
2. **Evaluate.** Each variation is scored with `_score_features()` against the live context `ctx` (onset, energy, bands, intensity, melody direction, vocal presence/drum fill/tension/harmonic/chord change, etc.). The score terms are (i) category ↔ instrument/feature affinity (bursts prefer strong onsets, trills track hi-hats, holds prefer sustained vocals, etc.), (ii) melody- direction match (ascending patterns match a rising melody; wrong-way patterns are penalized), (iii) intensity-appropriate length, (iv) diversity penalty for recently used patterns, (v) natural flow via a transition matrix, (vi) fatigue for consecutive same-category patterns, and (vii) extended-feature-vector affinity.
3. **Shortlist and pick.** The top 3 are kept and one is drawn via softmax with temperature 0.5. Even for identical inputs the same pattern does not always appear, keeping the chart fresh.

Each selection records the candidate count, shortlist, score, and applied transform in the `last_selection` diagnostic, explaining **"why this pattern came here."**

On the relationship between softmax sampling and explainability (review response).

Because the final selection is made by softmax sampling among the top-K, this stage depends partly on randomness. This conflicts with a strong sense of explainability in which "every decision reduces to deterministic rules." Yet the explainability of the system is substantively preserved for two reasons. First, the RNG is seeded (`random.Random(seed)`), so **the output is fully deterministically reproducible for the same input and seed**, and softmax is not a "black box" but sampling at a specified temperature (0.5) over a specified score distribution. Second, the explanation is provided *post-hoc* — the diagnostic records **the actually selected candidate and its score and competitors** regardless of whether it was an argmax or a sample, so for every note one can always trace "what was selected, and on

what grounds." That said, "why this candidate rather than the argmax (the specific realization of the sampling)" reduces only to the seed state and not to musical rules. We therefore lower the strength of the claim and characterize it as **"reproducible, post-hoc traceable explainability with a probabilistic variation."** For use cases requiring deterministic behavior, setting the softmax temperature to 0 (argmax) removes the sampling and restores full deterministic explainability.

5.3 Contour-Based Motif Memory and Section Replay

A human charter hears a 4-bar phrase, designs a lane figure for it, and when the same melody returns does not redesign from scratch but **varies** it. This is modeled at two levels.

Motif (MotifGenerator). The melody is quantized into 12 buckets and stored as an **interval contour** of successive intervals rather than absolute pitch. For example, the contour (+ 2, + 1, + 2) of one phrase matches another phrase in a different key. Matching is attempted at lengths 8, 6, 4 (longest first), with fuzzy matching allowing up to one interval mismatch, frequency-weighted priority, and cross-section persistence of memory. On replay, progressive transforms (identity → shift → mirror → reverse) are applied according to usage count, producing gradual reinterpretation like 1234 → 1234 → 1243 → 4321 .

Section replay (SectionReplay). The actual lane sequence placed in each section occurrence is recorded, and when the same section recurs, **~70% of the previous occurrence is reused verbatim** and only ~30% is varied in contiguous blocks (mirror/shift/reverse). Block-wise variation eliminates the fragmented replays produced by the old per-note random dropout, making most of the section immediately recognizable while a few phrases feel intentionally different.

5.4 Closed-Loop Density Controller

The intensity curve says "the chorus should be 90% dense," but if the audio is quiet the placement threshold is never met and density collapses, while if everything is loud it saturates. `DensityPlanner` adds a feedback loop.

1. Compute the **target NPS** at time t from $\text{intensity} \times \text{per-difficulty peak NPS}$.
2. Track the **actual NPS** in a 2-second sliding window.
3. Exponentially smooth the target/actual ratio to output a **correction factor** to multiply the placement threshold (under-dense → ↑ accept, over-dense → ↑ reject; clamped to 0.5–1.6).
4. In post-processing, trim only weak, non-downbeat, non-hold notes in windows exceeding 130% of the target.

As a result, a smooth difficulty arc is maintained even for quiet songs.

5.4.1 Phantom-Note Prevention — Resolving the Conflict with Rhythmic Fit (review response)

The fact that the density controller lowers the threshold in under-dense regions carries a risk of "phantom notes" — notes force-placed into quiet regions with no real attack — that would damage the top-priority condition of §1.1, **rhythmic fit**. The system defends against this with **multiple safeguards, not a single threshold manipulation**.

1. **The placement signal itself is onset/energy-based.** The placement condition is `contour ≥ threshold`, where $\text{contour} = 0.42 \text{ onset} + 0.18 \text{ energy} + \dots$ is onset-dominated. The density controller only *lowers the threshold* and cannot create contour, so a point with no signal ($\text{contour} \approx$

o) cannot pass no matter how low the threshold is. That is, the controller does not *inject* notes; it only modulates whether an already- present weak signal passes.

2. **Onset floor (explicit safeguard).** The *boost* direction of the correction factor is allowed only when a real transient exists. If the onset is below the floor `MIN_ONSET_FOR_BOOST = 0.15`, the correction is clamped to `min(correction, 1.0)`, **categorically blocking any threshold-lowering correction** (the suppression direction, i.e. $\text{correction} < 1$, is still allowed). Thus the path by which the density target would lower the threshold and fabricate phantom notes in silent/sustained regions is removed.
3. **Energy floor.** Independently of the above, any frame with `energy < 0.08` that is not a downbeat is unconditionally not placed (`place = False`). Intros and outros are additionally suppressed probabilistically.
4. **Asymmetric closed loop.** The boost correction is capped at $1.6\times$, so it can never drive the threshold to zero, and the post-pass `_density_trim` only *removes* (downbeat and hold notes are preserved). As a result, the "fill density" direction is strictly limited and only the "reduce density" direction is free.

With this layered defense, the density controller **raises density only where a real sound exists and cannot intervene in silence**. In the §6.3 regression metrics, the fact that hold–tap overlaps are always 0 and grid alignment stays at 1.0 supports that rhythmic fit is preserved while meeting the density target. Consequently the rhythmic-fit condition of §1.1 and the density control of §5.4 do not conflict.

5.5 Hand-Aware Lane Placement

`pick_lane()` blends three signals by style weights: (1) instrument anchoring (kick → outer lanes, snare → inner), (2) melody contour, and (3) movement patterns (trills, zigzags, stairs). On top of this it applies left-right hand alternation bias (`HandState`), anti-jack (avoids the same lane consecutively with 82% probability), triple-jack prevention, and hand-crossing suppression to produce physically natural hand movement. Lanes are provided with a per-game color scheme (DJMAX style) for all of 4–8 keys.

5.6 Humanizer

To soften mechanical perfection, a Gaussian jitter with standard deviation 8 ms is added to each note time, and the weight is randomly scaled by $\pm 7\%$. Downbeat notes are preserved so as not to harm rhythmic fit.

6. Evaluation and Auto-Improvement

6.1 Six-Dimensional Quality Evaluator

`evaluate_chart()` scores a chart on the following six dimensions (each 0–100) and takes a weighted sum.

Dimension	Weight	Measurement
Rhythmic fit	0.20	fraction of notes aligned to the 16th/triplet grid
Hand movement	0.20	mean lane jump (ideal 0.8–2.2), jack/crossing penalties
Pattern diversity	0.15	unique-ratio of lane 3-grams/2-grams
Repetition balance	0.15	4-gram repetition rate (sweet spot 15–40%)
Long notes	0.10	fit of the hold ratio to the per-difficulty target
Difficulty fit	0.20	fit of the NPS to the per-difficulty target range

Each dimension penalizes deviations from a per-difficulty target range (e.g. NPS 3.0–9.0 and hold ratio 5–22% for hard).

6.2 Multi-Seed Auto-Improvement

`auto_improve()` applies the "generate–evaluate–select" strategy at the **whole-chart level**. It generates several candidates with different seeds (so the pattern search, motif transforms, and humanization diverge), scores each on the six dimensions, and selects the highest total. It stops early if a candidate exceeds a threshold (75 by default). The selected chart is annotated with a **traceable generation report** containing the candidate comparison, chosen seed, score range, and strongest dimension, so one can see why the winner won.

6.3 Regression-Test Metrics

`tests/metrics.py` computes audio-independent objective metrics (note count, NPS, hold ratio, diversity, repetition rate, mean hand-movement distance, max jack run, max burst NPS, grid-alignment rate, hold–tap overlap count) to monitor regressions against a baseline (`baseline.json`). In particular, hold–tap overlap is a consistency constraint that must always be 0. Genre validation (`run_genre_validation.py`) and the quality tests (`run_quality_tests.py`) check that the metrics stay within target ranges across songs of differing character (EDM, ballad, rock, etc.).

7. Implementation and Deployment

The system is implemented in **pure Python 3 + NumPy** with no GPU or deep-learning framework, running on a laptop-class CPU. YouTube input is fetched as the best audio stream with `yt-dlp` and converted to WAV with a bundled `ffmpeg` (imageio-ffmpeg).

- **CLI (`chartgen.py`)**: a single command turns WAV/YouTube into a JSON/CSV chart.
- **Web app**: FastAPI backend + React/Vite frontend, with Docker and Cloudflare Pages deployment configs (`docker-compose.prod.yml`, `functions/api`).
- **Desktop app**: a single PyInstaller exe. On launch it starts a local server and opens an address-bar-less dedicated window via Edge/Chrome `--app` mode. No Python/ffmpeg/Node installation is required, and the exe is distributed as a GitHub Releases asset due to its size.

The output JSON has a `metadata` structure (bpm, beat_offset, keys, difficulty, and auto_improve's generation report) and a `notes` structure (time, lane, kind, duration, beat, weight), and can be converted to various formats by adding a game-specific adapter.

8. Discussion and Limitations

8.1 Scope and Limits of Explainability

Because every decision is based on explicit rules and feature scores, the user can tune parameters directly and inspect the rationale of each pattern/candidate selection via diagnostics. This is a clear advantage over black-box learned models. However, as discussed in §5.2, the softmax sampling in the final pattern selection means the system's decisions depend partly on (seeded) randomness. Hence the explainability of the system is not the strong sense in which "every choice reduces to deterministic rules," but a limited explainability that is **(i) fully reproducible under a fixed seed, (ii) always post-hoc traceable to the actual selection and its rationale, but (iii) whose specific sampling realization is explained only by the RNG state.** Where full determinism is required, setting the softmax temperature to 0 (argmax) removes this limit, at the cost of reduced chart diversity. This is an explicit trade-off between explainability and generative diversity.

8.2 Limits of Self-Evaluation and Threats to Validity

The six-dimensional evaluation of §6 and the objective metrics of §6.3 are all **self-evaluations** that measure how well the researcher-designed algorithm follows the researcher-defined rules (target NPS, hold ratio, hand-movement range, etc.). This demonstrates rule conformance only; it does **not guarantee construct validity** as to whether the generated charts are *perceived* as "good charts" by human players, or are comparable to expert charters' work. In particular, (i) the target ranges of the evaluation metrics are themselves hand-tuned, so evaluation and generation share the same assumptions (circularity), and (ii) there is no blind A/B comparison against human charts or user-preference study. Therefore the strong claim of "reproducing expert-chapter quality" is beyond the empirical scope of this paper, and the verified contribution of this work is limited to **achieving conformance to predefined rules.** Human-evaluation-based validation is left as future work (§8.3).

8.3 Other Limitations

(1) Band separation is an approximation rather than true source separation, so instrument classification can be inaccurate in complex mixes. (2) Beat/section detection assumes strong rhythm and is less accurate on ambient, classical, or irregular-meter songs. (3) The rule weights are hand-tuned from general principles observed across several commercial games; validation against large-scale user-preference data is future work. (4) Game-specific judgment rules for holds and chords must be handled separately in an adapter layer.

Ethical consideration. The system does not learn or copy any individual charter's work. Instead it codifies design principles commonly perceived as "good charting" across several games. This is a safe approach with respect to copyright and creative attribution.

Future work. We plan to introduce true source separation (a lightweight neural network), automatic tuning of rule weights from user play logs, expanded game-specific format adapters, a quantitative human-player-preference evaluation of generated charts (a user study), and a **controlled comparison against learning-based generators such as DDC [1] on the same hardware and songs** (measuring generation latency, resource usage, and human-judged quality together). Table 1 is only a paradigm-level resource comparison and does not substitute for such controlled experiments.

9. Conclusion

This paper proposed the system, a rule-based approach that automatically generates rhythm-game charts from arbitrary audio. It estimates a song's rhythm, structure, instruments, and musical character with a lightweight MIR pipeline, **encodes** expert-chart design principles **as rules** through feature-based pattern scoring, contour motif memory, closed-loop density control, and hand-aware placement, and selects the best chart via six-dimensional rule-conformance evaluation and multi-seed search. The quantitative experiments (§6.3) showed that all of 25 checks across 5 songs $\times$ 5 difficulties satisfy the target density, alignment, and overlap constraints (mean quality score 91.4, hold-tap overlap 0, grid alignment 1.0). The system's behavior is reproducible under a fixed seed and post-hoc traceable (limited explainability, §8.1), and it runs on a laptop-class CPU with no GPU, deep learning, or training data. That said, what this work demonstrates is **conformance to predefined rules**; the reproduction of human-perceived quality is future work to be validated via qualitative evaluation (§8.2). Within this scoped framing, the proposed system contributes a practical, transparent, and reproducible approach to automatic chart generation for rhythm games, adding a content-creation-oriented tool to the procedural generation of music-game content.

Code and Data Availability

The source code, sample data, and both the Korean and English versions of this paper are released under the MIT license; the public repository URL is withheld here to preserve anonymous review and will be provided in the camera-ready version. All results in this paper are reproducible with the procedure in Appendix A.

References

[1] C. Donahue, Z. C. Lipton, and J. McAuley, "Dance Dance Convolution," in *Proc. 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1703.06891.

Appendix A. Reproducibility

```
git clone <repository-url-withheld-for-anonymous-review>
cd rhythm-chart-generator
python -m pip install numpy yt-dlp
# local WAV -> hard-difficulty 4-key chart
python chartgen.py "song.wav" --keys 4 --difficulty hard --out chart.json --csv chart.csv
```

It reproduces deterministically for the same seed; changing `--seed` yields differently-feeling patterns for the same song.

Appendix B. Source Layout

File	Role
backend/audio_pipeline.py	audio analysis (§4)
backend/chart_engine.py	chart generation engine (§5)
backend/chart_evaluator.py	6-dimension evaluation & auto-improvement (§6)
backend/pattern_loader.py , backend/patterns/*.json	pattern library & transition matrix
backend/tests/metrics.py	regression metrics (§6.3)
chartgen.py	CLI entry point
backend/main.py , frontend/	web app
backend/desktop_app.py	desktop app launcher